

World-Frame Algorithm for First-Order Inverse Dynamics Derivatives with Constraint Embedding

Daniel J. Volpi and Patrick M. Wensing

Abstract—This supplement derives a world-frame variant of the first-order inverse dynamics derivative algorithm presented in the companion paper [1]. The body-frame Algorithm 1 of that paper contains an inner while loop that walks from each cluster to the root, applying a force transform at each step. Expressing all spatial quantities in the world frame can eliminate this inner loop: the off-diagonal derivative blocks for each cluster can then be formed collectively via matrix operations. This approach has been previously used for the special case of open chains [2]. For constraint-embedded systems, however, the cluster-to-cluster spatial force transform has additional structure (all forces from a child cluster accumulate onto a single output body in the parent) and this structure requires modification to the previously presented world-frame algorithm [2]. This document develops those modifications.

I. BACKGROUND

A. Notation

We roughly follow the companion papers [1] and [3], though with higher fidelity to [3] due to the emphasis on cluster semantics herein. We default to the nomenclature in the code to assist with its reading. By contrast, [3] more closely matches nomenclature used previously in the literature.

We assume that a kinematic connectivity graph with corresponding spanning tree $\mathcal{G}_S$ and cluster tree $\mathcal{G}_C$ has been constructed as in [3], and that the bodies have been numbered to satisfy the topological ordering constraints of [3] (Section III-B). We use k and ℓ for cluster indices, and i and j for body indices. We denote $\lambda(k)$ as the parent of cluster k in the cluster tree $\mathcal{G}_C$. C_k is the set of body numbers in cluster k , with $n_{b,k} = |C_k|$ and $C_k(j)$ denoting the j -th body (ordered by body number). The output body $\Lambda(k) \in C_{\lambda(k)}$ is the greatest spanning-tree ancestor in the parent cluster $\lambda(k)$ as shared by all bodies in cluster k ; equivalently, $\Lambda(k) = C_{\lambda(k)}(j^*)$, where $j^* = j^*(k)$ is the parent subindex giving the position of $\Lambda(k)$ within the parent cluster's body list. The output body of a cluster is called the parent body of an aggregate body (i.e., cluster) in [3].

We adopt the subtree notation of [2]: ν_k denotes the set of clusters in the subtree rooted at cluster k (including k), and $\bar{\nu}_k = \nu_k \setminus \{k\}$. When used as a block-row or block-column index, ν_k refers to all velocity DoFs of every cluster in ν_k . The velocity DoFs of cluster k alone are written as index $[k]$. Thus $\frac{\partial \tau}{\partial \mathbf{q}}[\nu_k, \ell]$ denotes the rectangular block whose rows span the DoFs of ν_k and whose columns span the DoFs of cluster ℓ .

Sans-serif symbols (e.g., S_k , M_k) denote cluster-level quantities, which are block-stacked over the $n_{b,k}$ bodies of cluster k ; upright bold symbols (e.g., $\mathbf{M}_i$) denote corresponding 6D body-level spatial vector quantities. A superscript 0 denotes a world-frame quantity, e.g., $\mathbf{S}_i^0$.

B. Prior results

As given in [2], the derivatives of inverse dynamics for an open-chain model can be expressed using world-frame quantities. For each body i , we consider

$$\mathbf{t}_1[i] = \check{\mathbf{M}}_i^0 \mathbf{S}_i^0, \quad (1)$$

$$\mathbf{t}_2[i] = \check{\mathbf{B}}_i^0 \mathbf{S}_i^0 + \check{\mathbf{M}}_i^0 \dot{\mathbf{Y}}_i^0, \quad (2)$$

$$\mathbf{t}_3[i] = \check{\mathbf{B}}_i^0 \check{\mathbf{\Psi}}_i^0 + \check{\mathbf{M}}_i^0 \check{\mathbf{\Psi}}_i^0 + (\check{\mathbf{f}}_i^0 \bar{\times}^*) \mathbf{S}_i^0, \quad (3)$$

$$\mathbf{t}_4[i] = (\check{\mathbf{B}}_i^0)^\top \mathbf{S}_i^0. \quad (4)$$

Here, $\check{\mathbf{f}}_i^0$, $\check{\mathbf{M}}_i^0$, $\check{\mathbf{B}}_i^0$ are composite (subtree-accumulated) world-frame forces, inertias, and Coriolis factors [2]; $\mathbf{S}_i^0$, $\check{\mathbf{\Psi}}_i^0$, $\check{\mathbf{\Psi}}_i^0$ are the world-frame motion-subspace and associated kinematics matrices of body i ; and $\dot{\mathbf{Y}}_i^0 = \dot{\mathbf{S}}_i^0 + \check{\mathbf{\Psi}}_i^0$ combines the body-frame and parent-frame rates of the motion subspace (see [2] for additional description). The notation $\mathbf{t}_1[i]$ denotes the $6 \times n_i$ block of $\mathbf{t}_1$ corresponding to the i -th joint. The four derivative blocks are then:

$$\frac{\partial \tau}{\partial \mathbf{q}}[\nu_i, i] = \mathbf{t}_4[\nu_i]^\top \check{\mathbf{\Psi}}_i^0 + \mathbf{t}_1[\nu_i]^\top \check{\mathbf{\Psi}}_i^0, \quad (5)$$

$$\frac{\partial \tau}{\partial \mathbf{q}}[i, \bar{\nu}_i] = (\mathbf{S}_i^0)^\top \mathbf{t}_3[\bar{\nu}_i], \quad (6)$$

$$\frac{\partial \tau}{\partial \dot{\mathbf{q}}}[\nu_i, i] = \mathbf{t}_4[\nu_i]^\top \mathbf{S}_i^0 + \mathbf{t}_1[\nu_i]^\top \dot{\mathbf{Y}}_i^0, \quad (7)$$

$$\frac{\partial \tau}{\partial \dot{\mathbf{q}}}[i, \bar{\nu}_i] = (\mathbf{S}_i^0)^\top \mathbf{t}_2[\bar{\nu}_i]. \quad (8)$$

These formulas do not directly generalize to constraint-embedded cluster trees. Every $\mathbf{t}_1[i]$, for example, has exactly 6 rows in the open-chain case, so stacking columnwise over a subtree yields a uniform $6 \times n_{\nu_i}$ matrix on which the products (5)–(8) are defined. For a cluster k with multiple bodies, the analogous block $\check{\mathbf{M}}_k^0 \mathbf{S}_k^0$ has $6n_{b,k}$ rows, which can vary across clusters and breaks this uniform stacking. Section II introduces accumulation operations to resolve this issue.

C. World-Frame Quantities

For a cluster k with $n_{b,k}$ bodies, the full motion transform from the world frame to the cluster frames is the $6n_{b,k} \times 6$ matrix

$$c_k \mathbf{X}_0 = \begin{bmatrix} c_k^{(1)} \mathbf{X}_0 \\ \vdots \\ c_k^{(n_{b,k})} \mathbf{X}_0 \end{bmatrix}, \quad (9)$$

where $c_k^{(j)} \mathbf{X}_0 \in \mathbb{R}^{6 \times 6}$ is the spatial motion transform from world frame to the local frame of body $C_k(j)$.

In practice, world-frame quantities are computed per body by extracting the 6×6 block ${}^{C_k(j)}\mathbf{X}_0$ from (9) for $j = 1, \dots, n_{b,k}$ and applying the inverse transform to body-level quantities. For example, writing $i = C_k(j)$ for the corresponding body:

$$\mathbf{S}_{k,i}^0 = ({}^i\mathbf{X}_0)^{-1} \mathbf{S}_{k,i}, \quad (10)$$

$$\mathbf{v}_i^0 = ({}^i\mathbf{X}_0)^{-1} \mathbf{v}_i, \quad (11)$$

$$\mathbf{f}_i^0 = {}^i\mathbf{X}_0^\top \mathbf{f}_i, \quad (12)$$

$$\mathbf{M}_i^0 = {}^i\mathbf{X}_0^\top \mathbf{M}_i {}^i\mathbf{X}_0. \quad (13)$$

Here $\mathbf{S}_{k,i} \in \mathbb{R}^{6 \times n_k}$ denotes the 6-row block of the cluster motion subspace $\mathbf{S}_k$ corresponding to body i (distinct from the spanning-tree joint subspace of tree joint i alone), while $\mathbf{v}_i, \mathbf{f}_i, \mathbf{M}_i$ are the standard 6-vector or 6×6 spatial quantities (velocity, force, and inertia) for body i . The same block-extraction and world-frame transformation applies to $\check{\Psi}_{k,i}^0, \check{\Psi}_{k,i}^0, \check{\Upsilon}_{k,i}^0$ (world-frame blocks of $\check{\Psi}_k, \check{\Psi}_k, \check{\Upsilon}_k$). One other common quantity is required:

$$\mathbf{B}_i^0 = (\mathbf{v}_i^0 \times^*) \mathbf{M}_i^0 - \mathbf{M}_i^0 (\mathbf{v}_i^0 \times) + (\mathbf{M}_i^0 \mathbf{v}_i^0) \bar{\times}^*. \quad (14)$$

II. REMOVING TRANSFORMS IN WORLD-FRAME COMPUTATIONS

The entire modification relative to [2] boils down to one main idea: a cluster with multiple bodies doesn't hand off a single 6-vector or 6×6 block when transformed to its parent – it hands off a sum over its bodies, landing in one slot (the output body) of the parent cluster. Everything below is the bookkeeping needed to make that summing precise in world-frame coordinates. For notational simplicity in what follows, we let $\ell = \lambda(k)$ and $j^* = j^*(k)$.

A. Inner products and the block-row sum

Off-diagonal derivative blocks require inter-cluster products of the form

$$\mathbf{S}_\ell^\top {}^k\mathbf{X}_\ell^\top \check{\mathbf{M}}_k \mathbf{S}_k, \quad (15)$$

where ${}^k\mathbf{X}_\ell^\top$ transforms cluster- k force quantities into the local frames of cluster ℓ . Following [3] (Eq. (36)), ${}^k\mathbf{X}_\ell^\top$ has a single non-zero block row since each body in cluster k connects through the output body $\Lambda(k)$ at position j^* in cluster ℓ . This gives the structure

$$\mathbf{S}_\ell^\top {}^k\mathbf{X}_\ell^\top = \mathbf{S}_{\ell, \Lambda(k)}^\top \begin{bmatrix} {}^{C_k(1)}\mathbf{X}_{\Lambda(k)}^\top & \dots & {}^{C_k(n_{b,k})}\mathbf{X}_{\Lambda(k)}^\top \end{bmatrix}, \quad (16)$$

When all quantities are expressed in the world frame, the 6×6 spatial transforms reduce to the identity.

As a result, the operation $\mathbf{S}_\ell^\top {}^k\mathbf{X}_\ell^\top \check{\mathbf{M}}_k \mathbf{S}_k$ can be re-written:

$$\mathbf{S}_\ell^\top {}^k\mathbf{X}_\ell^\top \check{\mathbf{M}}_k \mathbf{S}_k = (\mathbf{S}_{\ell, \Lambda(k)}^0)^\top \text{BlockRowSum}(\check{\mathbf{M}}_k^0 \mathbf{S}_k^0). \quad (17)$$

where for any matrix $\mathbf{F} \in \mathbb{R}^{6n_{b,k} \times m}$ the operation

$$\text{BlockRowSum}(\mathbf{F}) = \sum_{j=1}^{n_{b,k}} \mathbf{F}[j] \in \mathbb{R}^{6 \times m} \quad (18)$$

sums the $6 \times m$ blocks $\mathbf{F}[j]$ of $\mathbf{F}$. This block-row sum captures the aggregation structure of the cluster-level force transform

and allows the product to be computed without any transforms when all quantities are in the world frame.

B. Inertia propagation and the block-diagonal sum

Composite inertia propagates from cluster k to cluster ℓ via the congruence transform

$$\check{\mathbf{M}}_\ell += {}^k\mathbf{X}_\ell^\top \check{\mathbf{M}}_k {}^k\mathbf{X}_\ell. \quad (19)$$

The sparsity of ${}^k\mathbf{X}_\ell^\top$ (only its j^* -th block row is nonzero) means (19) affects only the j^* -th diagonal block of $\check{\mathbf{M}}_\ell$ according to

$$\check{\mathbf{M}}_\ell[j^*] += \sum_{i=1}^{n_{b,k}} {}^{C_k(i)}\mathbf{X}_{\Lambda(k)}^\top \check{\mathbf{M}}_k[i] {}^{C_k(i)}\mathbf{X}_{\Lambda(k)}$$

When all spatial quantities are expressed in the world frame, the 6×6 transforms reduce to the identity. In this case, the congruence amounts to summing the $n_{b,k}$ diagonal 6×6 blocks of $\check{\mathbf{M}}_k^0$ into the appropriate slot:

$$\check{\mathbf{M}}_\ell[j^*] += \text{BlockDiagSum}(\check{\mathbf{M}}_k^0), \quad (20)$$

where for any matrix $\mathbf{M} \in \mathbb{R}^{6n_{b,k} \times 6n_{b,k}}$ the operation

$$\text{BlockDiagSum}(\mathbf{M}) = \sum_{j=1}^{n_{b,k}} \mathbf{M}[j] \in \mathbb{R}^{6 \times 6}. \quad (21)$$

with $\mathbf{M}[j]$ denoting the j -th 6×6 diagonal block of $\mathbf{M}$. Note that for open-chain systems, both the `BlockRowSum` and `BlockDiagSum` reduce to identity operations.

III. UPDATED DERIVATIVE BLOCKS

A. Per-body temporaries and system-level accumulators

Mirroring the temporaries in (1)-(4), for each body $i \in C_k$, define the four $6 \times n_k$ matrices

$$\mathbf{F}_{1,i} = \check{\mathbf{M}}_i^0 \mathbf{S}_{k,i}^0, \quad (22)$$

$$\mathbf{F}_{2,i} = \check{\mathbf{B}}_i^0 \mathbf{S}_{k,i}^0 + \check{\mathbf{M}}_i^0 \check{\Upsilon}_{k,i}^0, \quad (23)$$

$$\mathbf{F}_{3,i} = \check{\mathbf{B}}_i^0 \check{\Psi}_{k,i}^0 + \check{\mathbf{M}}_i^0 \check{\Psi}_{k,i}^0 + (\check{\mathbf{f}}_i^0 \bar{\times}^*) \mathbf{S}_{k,i}^0, \quad (24)$$

$$\mathbf{F}_{4,i} = (\check{\mathbf{B}}_i^0)^\top \mathbf{S}_{k,i}^0, \quad (25)$$

Because all quantities share the world frame, no transform is needed when summing $\mathbf{F}_{m,i}$ contributions across bodies. Four system-level accumulator matrices $\mathbf{F}_m \in \mathbb{R}^{6 \times n}$, $m = 1, 2, 3, 4$, are then stored with block-column entries for cluster k as:

$$\mathbf{F}_m[k] = \sum_{i \in C_k} \mathbf{F}_{m,i} \quad (26)$$

B. Diagonal blocks

Considering equations (62) and (63) of [1], the diagonal blocks for cluster k are computed using world frame quantities

TABLE I: Body-frame vs. world-frame algorithm structure

Feature	Body frame (Alg. 1 of [1])	World frame (Alg. 1 Herein)
Temporaries	$\mathbf{t}_1\text{--}\mathbf{t}_4$ (body coords)	$\mathbf{F}_{1,i}\text{--}\mathbf{F}_{4,i}$ (world coords)
Inner while loop	Yes, ${}^\ell\mathbf{X}_{\lambda(\ell)}^\top$ per step	Eliminated via accumulation in temporaries
Off-diag. computations per cluster	One pair per ancestor	One pair, using ν_k columns
Inertia prop.	${}^k\mathbf{X}_{\lambda(k)}^\top \check{\mathbf{M}}_k {}^k\mathbf{X}_{\lambda(k)}$	BlockDiagSum into $[j^*]$ slot

as:

$$\frac{\partial \tau}{\partial \mathbf{q}}[k, k] = \sum_{i \in \mathcal{C}_k} \left(\mathbf{F}_{1,i}^\top \ddot{\Psi}_{k,i}^0 + \mathbf{F}_{4,i}^\top \dot{\Psi}_{k,i}^0 \right) + (\nabla_{\mathbf{q}_k} \mathbf{S}_k^\top \mathbf{F})|_{\mathbf{F}=\mathbf{f}_k}, \quad (27)$$

$$\frac{\partial \tau}{\partial \dot{\mathbf{q}}}[k, k] = \sum_{i \in \mathcal{C}_k} \left(\mathbf{F}_{1,i}^\top \dot{\Upsilon}_{k,i}^0 + \mathbf{F}_{4,i}^\top \mathbf{S}_{k,i}^0 \right). \quad (28)$$

The last term in (27) is nonzero only for clusters with configuration-dependent $\mathbf{S}_k$.

C. Off-diagonal blocks with the parent

We next express equations (61), (62), (64), and (65) of [1] using world-frame quantities, while employing accumulated quantities to address the removal of inter-cluster transforms.

Let $\ell = \lambda(k)$ and $i = \Lambda(k)$. After accumulating (26), the four off-diagonal blocks are:

$$\frac{\partial \tau}{\partial \mathbf{q}}[\nu_k, \ell] = \mathbf{F}_1[\nu_k]^\top \ddot{\Psi}_{\ell,i}^0 + \mathbf{F}_4[\nu_k]^\top \dot{\Psi}_{\ell,i}^0, \quad (29)$$

$$\frac{\partial \tau}{\partial \mathbf{q}}[\ell, \nu_k] = (\mathbf{S}_{\ell,i}^0)^\top \mathbf{F}_3[\nu_k], \quad (30)$$

$$\frac{\partial \tau}{\partial \dot{\mathbf{q}}}[\nu_k, \ell] = \mathbf{F}_1[\nu_k]^\top \dot{\Upsilon}_{\ell,i}^0 + \mathbf{F}_4[\nu_k]^\top \mathbf{S}_{\ell,i}^0, \quad (31)$$

$$\frac{\partial \tau}{\partial \dot{\mathbf{q}}}[\ell, \nu_k] = (\mathbf{S}_{\ell,i}^0)^\top \mathbf{F}_2[\nu_k], \quad (32)$$

These revisions are collected into the world-frame algorithm in Algorithm 1.

IV. COMPARISON WITH THE BODY-FRAME ALGORITHM

Table I summarizes the key structural differences. The $\approx 33\%$ runtime reduction reported in the companion paper comes from eliminating the repeated transforms and replacing the inner while loop with a single pair of matrix products per cluster. For large clusters the BlockDiagSum/BlockRowSum propagation adds $O(n_{b,k})$ floating-point additions per cluster, which is modest compared to the prior cost of repeated transforms that increased in number with tree depth.

REFERENCES

- [1] D. J. Volpi and P. M. Wensing, “Adapting rigid-body dynamics derivatives for constraint embedding closed-chain models,” *IEEE Robotics and Automation Letters*, 2026, to appear.
- [2] S. Singh, R. P. Russell, and P. M. Wensing, “Efficient analytical derivatives of rigid-body dynamics using spatial vector algebra,” *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 1776–1783, 2022.
- [3] M. Chignoli, N. Adrian, S. Kim, and P. M. Wensing, “A propagation perspective on recursive forward dynamics for systems with kinematic loops,” *IEEE Transactions on Robotics*, vol. 41, pp. 5584–5603, 2025.

Algorithm 1 World-Frame First-Order Derivatives of ID

Require: $\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}$, model

1: $\mathbf{v}_0 = \mathbf{0}$; $\mathbf{a}_0 = -\mathbf{a}_g$; $\mathbf{F}_m = \mathbf{0}_{6 \times n}$ for $m = 1, 2, 3, 4$

2: **for** $k = 1$ **to** N **do**

3: $\mathbf{v}_k, \mathbf{a}_k, \dot{\Psi}_k, \dot{\Psi}_k, \dot{\Upsilon}_k$

4: ${}^{\mathcal{C}_k}\mathbf{X}_0$ via recursive kinematics.

5: **for each** $i \in \mathcal{C}_k$, using block ${}^i\mathbf{X}_0$ from ${}^{\mathcal{C}_k}\mathbf{X}_0$ **do**

6: $\mathbf{v}_i^0, \mathbf{S}_{k,i}^0, \dot{\Psi}_{k,i}^0, \ddot{\Psi}_{k,i}^0, \dot{\Upsilon}_{k,i}^0 \leftarrow ({}^i\mathbf{X}_0)^{-1}(\cdot)$

7: $\mathbf{f}_i^0 \leftarrow {}^i\mathbf{X}_0^\top \mathbf{f}_i$

8: $\check{\mathbf{M}}_i^0 \leftarrow (13)$; $\check{\mathbf{B}}_i^0 \leftarrow (14)$ using $\check{\mathbf{M}}_i^0, \mathbf{v}_i^0$

9: $\check{\mathbf{f}}_i^0 = \mathbf{f}_i^0$

10: **end for**

11: **end for**

12: **for** $k = N$ **to** 1 **do**

13: **for each** $i \in \mathcal{C}_k$ **do**

14: $\mathbf{F}_{1,i} = \check{\mathbf{M}}_i^0 \mathbf{S}_{k,i}^0$; $\mathbf{F}_{4,i} = (\check{\mathbf{B}}_i^0)^\top \mathbf{S}_{k,i}^0$

15: $\mathbf{F}_{2,i} = \check{\mathbf{B}}_i^0 \mathbf{S}_{k,i}^0 + \check{\mathbf{M}}_i^0 \dot{\Upsilon}_{k,i}^0$

16: $\mathbf{F}_{3,i} = \check{\mathbf{B}}_i^0 \dot{\Psi}_{k,i}^0 + \check{\mathbf{M}}_i^0 \ddot{\Psi}_{k,i}^0 + (\check{\mathbf{f}}_i^0 \bar{\times}^*) \mathbf{S}_{k,i}^0$

17: $\frac{\partial \tau}{\partial \mathbf{q}}[k, k] += \mathbf{F}_{1,i}^\top \ddot{\Psi}_{k,i}^0 + \mathbf{F}_{4,i}^\top \dot{\Psi}_{k,i}^0$

18: $\frac{\partial \tau}{\partial \dot{\mathbf{q}}}[k, k] += \mathbf{F}_{1,i}^\top \dot{\Upsilon}_{k,i}^0 + \mathbf{F}_{4,i}^\top \mathbf{S}_{k,i}^0$

19: $\mathbf{F}_m[k] += \mathbf{F}_{m,i}$ for $m = 1, 2, 3, 4$

20: **end for**

21: $\frac{\partial \tau}{\partial \mathbf{q}}[k, k] += \nabla_{\mathbf{q}_k} \mathbf{S}_k^\top \mathbf{F}|_{\mathbf{F}=\mathbf{f}_k}$

22: **if** $\lambda(k) > 0$ **then**

23: $\ell = \lambda(k)$; $i = \Lambda(k)$; $j^* = j^*(k)$

24: $\frac{\partial \tau}{\partial \mathbf{q}}[\nu_k, \ell] = \mathbf{F}_1[\nu_k]^\top \ddot{\Psi}_{\ell,i}^0 + \mathbf{F}_4[\nu_k]^\top \dot{\Psi}_{\ell,i}^0$

25: $\frac{\partial \tau}{\partial \mathbf{q}}[\ell, \nu_k] = (\mathbf{S}_{\ell,i}^0)^\top \mathbf{F}_3[\nu_k]$

26: $\frac{\partial \tau}{\partial \dot{\mathbf{q}}}[\nu_k, \ell] = \mathbf{F}_1[\nu_k]^\top \dot{\Upsilon}_{\ell,i}^0 + \mathbf{F}_4[\nu_k]^\top \mathbf{S}_{\ell,i}^0$

27: $\frac{\partial \tau}{\partial \dot{\mathbf{q}}}[\ell, \nu_k] = (\mathbf{S}_{\ell,i}^0)^\top \mathbf{F}_2[\nu_k]$

28: $\check{\mathbf{M}}_\ell^0[j^*] += \text{BlockDiagSum}(\check{\mathbf{M}}_k^0)$

29: $\check{\mathbf{B}}_\ell^0[j^*] += \text{BlockDiagSum}(\check{\mathbf{B}}_k^0)$

30: $\check{\mathbf{f}}_\ell^0[j^*] += \text{BlockRowSum}(\check{\mathbf{f}}_k^0)$

31: $\mathbf{f}_\ell += {}^k\mathbf{X}_{\lambda(k)}^\top \mathbf{f}_k$ [body-frame force, for $\nabla \mathbf{S}$ term]

32: **end if**

33: **end for**

34: **return** $\frac{\partial \tau}{\partial \mathbf{q}}, \frac{\partial \tau}{\partial \dot{\mathbf{q}}}$